Cuaderno 00 · Control de integridad e inventario de insumos y fuentes

Expediente: BES-jun · **Cuaderno:** 1 de 7 · **Estado del expediente:** Listo para revisión (no Completo)

1. Introducción y contexto (el problema)

Todos los cálculos del expediente parten de un archivo Excel de trabajo. Si ese archivo difiere del original recibido (por una edición manual, un guardado desde otra versión del programa, una copia equivocada o una corrupción), todo resultado posterior puede ser correcto respecto del archivo equivocado y nadie lo notaría. Este libro además contiene `=TODAY()` y fórmulas que se recalculan al abrirlo: abrirlo y guardarlo ya lo cambiaría.

2. Justificación y necesidad

Demostrar, antes de calcular nada: (a) que la copia de trabajo es idéntica byte a byte al original; (b) qué contiene el archivo (hojas, fórmulas, macros, enlaces externos, gráficos); (c) qué fuentes técnicas existen, para qué se usan y que siguen íntegras. Sin esa evidencia, un número del informe no tiene procedencia comprobable.

3. Objetivo general

Producir una verificación reproducible de identidad e inventario de insumos, con criterios de aceptación ejecutables que **detienen la ejecución** si alguno falla.

4. Dominio y alcance

Incluye: huellas SHA-256 y MD5 de tres rutas del mismo libro; control negativo (prueba de que la comparación puede fallar); estructura del contenedor `.xlsx`; inventario de fuentes técnicas con su papel; verificación de que los otros dos datasets no se abrieron.

No incluye: validar el contenido del libro (cuaderno 01); abrir o leer filas de `RB-1521` y `RB-IA-2191`; ejecutar macros del XLSM; modificar cualquier original.

5. Fuentes y datos (entradas y salidas)

Entradas: `entrada/BES_jun.xlsx`, `fuentes/originales/`, la copia de la raíz, `docs/inventario_fuentes.md` y `dataset.md` (huellas registradas). **Salidas:** tablas de huellas, veredicto de identidad, inventario del contenedor, inventario de fuentes y tabla de criterios.

6. Objetivos específicos y criterios de aceptación

C00-1 las tres copias coinciden en tamaño, SHA-256 y MD5 · **C00-2** el control negativo detecta un bit alterado · **C00-3** el contenedor no tiene macros, enlaces externos ni archivos corruptos, y tiene 5 hojas y 30 gráficos · **C00-4** el SHA-256 coincide con el de la ficha `dataset.md` · **C00-5** cada fuente técnica coincide con su huella registrada · **C00-6** los datasets en espera solo se comprobaron por existencia y tamaño.

7. Riesgo si este paso falla

Resultados de ingeniería sin procedencia: un PIP o un caudal calculado sobre un archivo alterado se vería idéntico a uno correcto. Una decisión operativa basada en él no sería auditable.

Preparación del entorno

Qué hace la celda siguiente. Localiza el expediente subiendo desde el directorio actual hasta la carpeta que contiene `dataset.md` y `entrada/`; así el cuaderno funciona se abra desde donde se abra y no depende de rutas escritas a mano. Después fija el directorio de trabajo, habilita el paquete `src/` e imprime la **ficha de reproducibilidad**: fecha, versiones de Python y bibliotecas, y la huella del insumo. Esa ficha permite saber, meses después, con qué entorno se produjo cada resultado.

Código

```
from pathlib import Path
import os, sys

def localizar_expediente() -> Path:
    # Sube desde el directorio actual hasta la carpeta que tiene dataset.md y entrada/.
    for p in [Path.cwd().resolve(), *Path.cwd().resolve().parents]:
        if (p / "dataset.md").is_file() and (p / "entrada").is_dir():
            return p
    raise RuntimeError("Ejecute el cuaderno dentro del expediente (donde están dataset.md y entrada/).")

RAIZ = localizar_expediente()                # .../datasets/en_uso/BES-jun
RAIZ_PROYECTO = RAIZ.parents[2]              # .../HUC
os.chdir(RAIZ)
sys.path.insert(0, str(RAIZ / "src"))

import pandas as pd
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 180)
pd.set_option("display.max_colwidth", None)
pd.set_option("display.float_format", lambda v: f"{v:,.8g}")

from analisis.criterios import Criterios
from analisis.entorno import ficha
from analisis.presentacion import T
T(pd.DataFrame(list(ficha().items()), columns=["campo", "valor"]))
```

Salida

campo	valor
fecha_ejecucion	2026-10-20T14:25:28-03:00
python	3.14.8
sistema	Linux 7.2.10-100.fc43.x86_64
pandas	3.0.6
numpy	2.5.4
matplotlib	3.11.2
openpyxl	3.1.5
expediente	BES-jun
insumo	entrada/BES_jun.xlsx
insumo_sha256	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85

Paso 1 · Qué copias existen y dónde

El libro existe en tres lugares: la **copia de trabajo** (la que leen el código y los cuadernos), el **original protegido** en `fuentes/originales/` (archivo de solo lectura) y el **archivo recibido en la raíz** del proyecto. La política del expediente es que solo se escribe sobre la copia de trabajo, y que el código ni siquiera la guarda (`openpyxl` abre y lee). El control consiste en comprobar que los tres son el mismo archivo.

Se calculan **dos huellas distintas** (SHA-256 y MD5) más el tamaño en bytes. Usar dos algoritmos no es redundancia estética: MD5 está registrado en documentos anteriores del proyecto y SHA-256 es el que se considera confiable frente a alteraciones; que ambos coincidan entre copias refuerza la lectura. Una coincidencia accidental de SHA-256 entre

archivos distintos es prácticamente imposible por azar.

Código

```
from ingesta.integridad import comparar

NOMBRE_ORIGINAL = "Copia de FINAL BES-jun.xlsx"
rutas = {
    "copia de trabajo (entrada/)":    RAIZ / "entrada" / "BES_jun.xlsx",
    "original (fuentes/originales/)": RAIZ_PROYECTO / "fuentes" / "originales" / NOMBRE_ORIGINAL,
    "recibido (raíz del proyecto)":   RAIZ_PROYECTO / NOMBRE_ORIGINAL,
}
veredicto = comparar(rutas)          # lee en bloques; no modifica nada
huellas_tabla = pd.DataFrame(veredicto["filas"])
print("Copias presentes:", int(huellas_tabla["presente"].sum()), "de", len(rutas))
print("Coinciden en bytes, SHA-256 y MD5:", "SÍ" if veredicto["coincide"] else "NO")
T(huellas_tabla[["copia", "bytes", "sha256", "md5"]])
```

Salida

```
Copias presentes: 3 de 3
Coinciden en bytes, SHA-256 y MD5: SÍ
```

Salida

copia	bytes	sha256	md5
copia de trabajo (entrada/)	329308	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85	ba89f317f505f321c496e44ab5046104
original (fuentes/originales/)	329308	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85	ba89f317f505f321c496e44ab5046104
recibido (raíz del proyecto)	329308	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85	ba89f317f505f321c496e44ab5046104

Cómo leer el resultado. Las tres filas deben mostrar el mismo tamaño y las mismas dos huellas. `comparar` está escrita para ser conservadora: **una ruta ausente no cuenta como aprobada** (el veredicto sería «NO»), porque un control que no pudo ejecutarse no demuestra nada.

Lo que esto prueba: que los tres archivos son idénticos bit a bit. **Lo que no prueba:** que el contenido sea correcto, completo o apto para decidir. Ese es el trabajo de los cuadernos 01 a 05.

Paso 2 · Control negativo: ¿puede este control fallar?

Un control de integridad que siempre dice «correcto» es peor que no tener control. La práctica de ingeniería de pruebas es demostrar que la prueba **detecta** el defecto que debe detectar. Aquí se lee el libro en memoria, se invierte **un solo bit** en la mitad del archivo y se recalculan las huellas. No se escribe nada en disco.

Código

```
from ingesta.integridad import control_negativo

negativo = control_negativo(rutas["copia de trabajo (entrada/)"])
T(pd.DataFrame(
    {
        "huella": ["SHA-256", "MD5"],
        "original": [negativo["sha256_original"], negativo["md5_original"]],
        "con 1 bit alterado": [negativo["sha256_alterado"], negativo["md5_alterado"]],
    }
))
```

Salida

huella	original	con 1 bit alterado
SHA-256	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85	220122f31fe0d75a1ffe7f8bbee6a2e320945ba0e4f47cf1a2dfdeccc12a5478
MD5	ba89f317f505f321c496e44ab5046104	4c0f2c328c9525f3bd3223ff2f589327

Código

```
print("Posición del byte alterado:", negativo["posicion_alterada"])
print("El control detecta la alteración de un solo bit:", "SÍ" if negativo["detecta_el_cambio"] else "NO")
```

Salida

```
Posición del byte alterado: 164654
El control detecta la alteración de un solo bit: SÍ
```

Interpretación. Con un único bit distinto, ambas huellas cambian por completo (no «un poco»). Eso es lo esperable de una función hash y demuestra que la igualdad observada en el Paso 1 es informativa: si la copia hubiera sido tocada, se vería.

Paso 3 · Qué contiene el archivo (estructura del contenedor)

Un `.xlsx` es un archivo ZIP con XML dentro. Mirarlo sin abrir las hojas responde preguntas de **riesgo** que condicionan todo el análisis: ¿hay macros que podrían ejecutar código? ¿hay vínculos a otros libros (valores que dependen de archivos que no tenemos)? ¿el ZIP está corrupto? ¿cuántas hojas y gráficos trae? La función `inventario_zip` lo contesta solo leyendo los nombres de las entradas y probando su CRC.

Código

```
from ingesta.estructura_libro import inventario_zip

contenedor = inventario_zip(rutas["copia de trabajo (entrada/)"])
T(pd.DataFrame(list(contenedor.items()), columns=["propiedad", "valor"]))
```

Salida

propiedad	valor
entradas_zip	182
hojas_xml	5
graficos	30
imagenes	1
tiene_macros_vba	no
tiene_enlaces_externos	no
tiene_cadena_de_calculo	sí
primer_archivo_corrupto	—

Interpretación. Sin macros VBA ni enlaces externos, el libro es **autocontenido**: sus valores dependen solo de lo que hay dentro, y abrirlo no ejecuta código. Existe `calcChain.xml`: significa que Excel guardó una cadena de cálculo, es decir, los valores calculados que leemos son los **valores en caché** de la última vez que Excel calculó. Esto importa: el cuaderno 02 contrasta el código contra esa caché, y por eso ese contraste reproduce la hoja, pero no es una validación independiente. Los 30 gráficos son objetos de presentación del libro; no se usan como datos.

Paso 4 · Fuentes técnicas y su papel

Los cálculos del libro se apoyan en fuentes: especificaciones de ecuaciones, un libro de fórmulas con macros, un PDF de diseño y un artículo. Para cada una se calcula la huella actual y se compara con la **registrada** en `docs/inventario_fuentes.md` el día del inventario. Si una fuente cambió, cualquier trazabilidad construida sobre ella queda en duda.

Dos reglas de este paso:

1. Los cuatro `.docx` de ecuaciones se comparan entre sí: si son el mismo archivo, cuentan como **una** fuente.
2. `RB-1521` y `RB-IA-2191` son datasets de otro expediente (en espera). **No se abren ni se leen sus bytes**: solo se consulta `os.stat` (existencia y tamaño) y se compara el tamaño con el registrado. Mezclarlos con BES-jun está prohibido.

Código

```
import os, re
from ingesta.integridad import huellas
```

```
# 1) Huellas registradas el día del inventario (se leen del documento, no se escriben a mano aquí).
doc = (RAIZ_PROYECTO / "docs" / "inventario_fuentes.md").read_text(encoding="utf-8")
registrado = {}
for linea in doc.splitlines():
    m = re.match(r"\| `([^`]+)` \| (\d+) \| `([0-9a-f]{64})` \| `([0-9a-f]{32})`", linea)
    if m:
        registrado[m.group(1)] = (int(m.group(2)), m.group(3), m.group(4))
base_e1 = registrado["Modelo_Sensorless_PIP_ESP_Set_Ecuaciones.docx"]
for n in range(1, 4):                       # la tabla dice "igual a E1" para los duplicados
    registrado[f"Modelo_Sensorless_PIP_ESP_Set_Ecuaciones ({n}).docx"] = base_e1

# 2) Papel de cada fuente en este expediente.
papel = {
    "Copia de FINAL BES-jun.xlsx": "Insumo: libro BES-jun",
    "Modelo_Sensorless_PIP_ESP_Set_Ecuaciones.docx": "E1: especificación algebraica",
    "Copia de Copia de Ecuaciones  _.xlsm": "X1: fórmulas de otro libro; tiene VBA, no se ejecuta",
    "paper_sensorless - v2.docx": "A1: contexto; sus métricas no son resultados de BES-jun",
    "004-ESPdesign.pdf": "R1: contexto de diseño (Hazen-Williams, p. 2)",
}
for n in range(1, 4):
    papel[f"Modelo_Sensorless_PIP_ESP_Set_Ecuaciones ({n}).docx"] = "Duplicado exacto de E1"

otro_expediente = {"RB-1521-PY-121120.xls", "RB-IA-2191-Data.xlsx"}
carpeta = RAIZ_PROYECTO / "fuentes" / "originales"
filas = []
for ruta in sorted(carpeta.iterdir()):
    if ruta.name == "trabajo.md":
        continue                                # el plan rector se actualiza; no se compara su huella
    if ruta.name in otro_expediente:
        esperado = registrado[ruta.name][0]
        tam = os.stat(ruta).st_size             # solo existencia y tamaño: sus bytes no se leen
        filas.append({"archivo": ruta.name, "papel": "Dataset de otro expediente (no se abre)",
                      "bytes": tam, "sha256 (16 primeros)": "no calculado",
                      "coincide con lo registrado": "tamaño sí" if tam == esperado else "tamaño NO"})
        continue
    h = huellas(ruta)
    reg = registrado.get(ruta.name)
    filas.append({"archivo": ruta.name, "papel": papel.get(ruta.name, "sin papel asignado"),
                  "bytes": h["bytes"], "sha256 (16 primeros)": h["sha256"][:16],
                  "coincide con lo registrado": "sí" if reg and (h["bytes"], h["sha256"], h["md5"]) == reg
else "NO"})
T(pd.DataFrame(filas), ancha=True)
```

Salida

archivo	papel	bytes	sha256 (16 primeros)	coincide con lo registrado
004-ESPdesign.pdf	R1: contexto de diseño (Hazen-Williams, p. 2)	320563	9d14c9db228f3365	sí
Copia de Copia de Ecuaciones _.xlsm	X1: fórmulas de otro libro; tiene VBA, no se ejecuta	142127	1e9eb80131a48abd	sí
Copia de FINAL BES-jun.xlsx	Insumo: libro BES-jun	329308	ccb5a738baea1332	sí
Modelo_Sensorless_PIP_ESP_Set_Ecuaciones (1).docx	Duplicado exacto de E1	43143	4ef527d9b2cafdb1	sí
Modelo_Sensorless_PIP_ESP_Set_Ecuaciones (2).docx	Duplicado exacto de E1	43143	4ef527d9b2cafdb1	sí
Modelo_Sensorless_PIP_ESP_Set_Ecuaciones (3).docx	Duplicado exacto de E1	43143	4ef527d9b2cafdb1	sí
Modelo_Sensorless_PIP_ESP_Set_Ecuaciones.docx	E1: especificación algebraica	43143	4ef527d9b2cafdb1	sí
RB-1521-PY-121120.xls	Dataset de otro expediente (no se abre)	84480	no calculado	tamaño sí
RB-IA-2191-Data.xlsx	Dataset de otro expediente (no se abre)	15491	no calculado	tamaño sí
paper_sensorless - v2.docx	A1: contexto; sus métricas no son resultados de BES-jun	34831	9eeddb2f15597218	sí

Código

```
docx = [f for f in filas if f["archivo"].startswith("Modelo_Sensorless")]
print("DOCX de ecuaciones:", len(docx), "| SHA-256 distintos entre ellos:", len({f["sha256 (16 primeros)"] for
f in docx}))
```

```
print("Fuentes con huella que NO coincide:", [f["archivo"] for f in filas if f["coincide con lo registrado"]
== "NO"] or "ninguna")
```

Salida

```
DOCX de ecuaciones: 4 | SHA-256 distintos entre ellos: 1
Fuentes con huella que NO coincide: ninguna
```

Interpretación. Todas las fuentes con huella coinciden con lo registrado, y los cuatro `.docx` son un único documento con cuatro nombres: la trazabilidad se apoya en **una** especificación E1, no en cuatro. Los datasets `RB-1521` y `RB-IA-2191` figuran solo con su tamaño; esto documenta que existen y que no se tocaron. Que el XLSM tenga VBA no es un problema mientras no se ejecute; se lee únicamente como texto de fórmulas.

El papel de cada fuente es una **decisión de alcance**, no una validación: que una fuente esté íntegra no significa que sus fórmulas sean aplicables a este libro (eso se revisa en el cuaderno 02).

Paso 5 · Contraste con la ficha del expediente

La ficha `dataset.md` es el documento que otra persona leerá primero. Si la huella escrita allí no coincidiera con el archivo real, la ficha estaría afirmando algo falso. Se extrae el SHA-256 de la ficha y se compara con el calculado en el Paso 1.

Código

```
ficha_md = (RAIZ / "dataset.md").read_text(encoding="utf-8")
sha_ficha = re.search(r"SHA-256: `([0-9a-f]{64})`", ficha_md).group(1)
sha_trabajo = huellas_tabla.loc[huellas_tabla["copia"].str.startswith("copia de trabajo"), "sha256"].iloc[0]
T(pd.DataFrame({"fuente": ["dataset.md", "archivo real"], "SHA-256": [sha_ficha, sha_trabajo]}))
```

Salida

fuente	SHA-256
dataset.md	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85
archivo real	ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85

Paso 6 · Criterios de aceptación (ejecutables)

Cada criterio declarado en la portada se evalúa aquí con la evidencia calculada arriba. Si alguno no se cumple, `exigir()` lanza un error y el cuaderno **no termina**: no se entrega un inventario con un control fallido.

Código

```
crit = Criterios("00_inventario_y_fuentes")
crit.comprobar("C00-1", "Tres copias idénticas (bytes, SHA-256, MD5)", veredicto["coincide"],
               f"{len(veredicto['filas'])} copias; ausentes: {veredicto['ausentes'] or 'ninguna'}")
crit.comprobar("C00-2", "El control negativo detecta un bit alterado", negativo["detecta_el_cambio"],
               "ambas huellas cambian")
crit.comprobar("C00-3", "Contenedor sin VBA, sin enlaces externos, sin corrupción, 5 hojas, 30 gráficos",
               (not contenedor["tiene_macros_vba"]) and (not contenedor["tiene_enlaces_externos"])
               and contenedor["primer_archivo_corrupto"] is None
               and contenedor["hojas_xml"] == 5 and contenedor["graficos"] == 30,
               f"hojas={contenedor['hojas_xml']}, gráficos={contenedor['graficos']}")
crit.comprobar("C00-4", "SHA-256 de dataset.md = SHA-256 del archivo", sha_ficha == sha_trabajo,
sha_trabajo[:16] + "…")
crit.comprobar("C00-5", "Fuentes técnicas coinciden con su huella registrada",
               all(f["coincide con lo registrado"] in ("sí", "tamaño sí") for f in filas),
               f"{sum(f['coincide con lo registrado'] == 'sí' for f in filas)} huellas completas + 2 tamaños")
crit.comprobar("C00-6", "Datasets en espera: solo existencia y tamaño",
               all(f["sha256 (16 primeros)"] == "no calculado" for f in filas if f["archivo"] in
otro_expediente),
               "RB-1521 y RB-IA-2191 no se leyeron")
T(crit.tabla())
```

Salida

id	criterio	resultado	evidencia
C00-1	Tres copias idénticas (bytes, SHA-256, MD5)	CUMPLE	3 copias; ausentes: ninguna
C00-2	El control negativo detecta un bit alterado	CUMPLE	ambas huellas cambian
C00-3	Contenedor sin VBA, sin enlaces externos, sin corrupción, 5 hojas, 30 gráficos	CUMPLE	hojas=5, gráficos=30
C00-4	SHA-256 de dataset.md = SHA-256 del archivo	CUMPLE	ccb5a738baea1332…
C00-5	Fuentes técnicas coinciden con su huella registrada	CUMPLE	8 huellas completas + 2 tamaños
C00-6	Datasets en espera: solo existencia y tamaño	CUMPLE	RB-1521 y RB-IA-2191 no se leyeron

Código

```
crit.exigir()
print("Todos los criterios de aceptación del cuaderno 00 se cumplen.")
```

Salida

```
Todos los criterios de aceptación del cuaderno 00 se cumplen.
```

Conclusión técnica

- **Resultado:** la copia de trabajo es idéntica al original y a la copia recibida; el contenedor es autocontenido (sin macros ni enlaces); las fuentes técnicas están íntegras; los datasets en espera no se tocaron.
- **Qué habilita:** los cuadernos 01 a 06 pueden afirmar que sus resultados provienen de **este** archivo, identificado por su SHA-256.
- **Limitaciones:** la identidad binaria no dice nada sobre la calidad del contenido. Si el archivo se vuelve a descargar, se renombra o se reabre y guarda en Excel, el control debe repetirse. La caché de valores calculados es del último cálculo de Excel; por eso los contrastes contra ella (cuaderno 02 y siguientes) no son validación independiente.
- **Siguiente paso:** cuaderno 01, inspección de estructura y calidad del contenido.

Relación con el sistema de reglas

Este cuaderno no produce parámetros para el motor. Comprueba que la copia de trabajo es el mismo archivo que se analizó. Sin esa identidad, `integracion/manifiesto.json` no tendría una huella que citar.

Limitaciones

- La huella SHA-256 demuestra **identidad binaria**, no que el contenido del libro sea correcto.
- El inventario de fuentes registra huellas de una fecha; si el documento de inventario estuviera mal, el contraste lo heredaría.
- Los archivos `RB-*` solo se contrastan con `os.stat` (tamaño y fecha): **no se abren** por la regla del expediente.
- La ficha `dataset.md` es un documento humano: se contrasta, no se da por cierta.

Referencias

Solo se listan las fuentes realmente consultadas en este cuaderno.

1. **Libro de trabajo:** `entrada/BES_jun.xlsx`, copia byte a byte de «Copia de FINAL BES-jun.xlsx» (SHA-256 `ccb5a738baea13321993e8b6eccefd6cac90ecda86b1c4b9181f897fcfa83f85`). Cinco hojas: QF-554H, QF-545H, QF-500H, QF-378HST y QF-549HST.
2. **`docs/inventario_fuentes.md`** (raíz del proyecto): registro de huellas de las fuentes en la fecha del inventario.

3. **`trabajo.md` y `trabajo2.md`** (raíz del proyecto): plan de trabajo, reglas de no mezclar libros y bloqueo P-14.
4. **Software:** Python 3.14.8, pandas 3.0.6, numpy 2.5.4, matplotlib 3.11.2, openpyxl 3.1.5 (versiones tomadas de la ficha de entorno de cada ejecución).